

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра математического обеспечения и применения ЭВМ

ОТЧЕТ
по практической работе №4
по дисциплине «Системы параллельной обработки данных»
Тема: Виртуальные топологии

Студент гр. 1306

Шаврин А.П.

Преподаватель

Татаринов Ю.С.

Санкт-Петербург

2025

Цель работы.

Изучить, как использовать виртуальные топологии в библиотеке MPI. Реализовать программу, использующую функцию `MPI_Cart_create` для создания виртуальной топологии как основную часть для решения задачи. Проанализировать работу программы.

Постановка задачи.

Вариант 6. Число процессов K кратно трем: $K = 3N$, $N > 1$ В процессах 0, 1 и 2 дано по N целых чисел. Определить для всех процессов декартову топологию в виде матрицы размера $N \times 3$, после чего, используя функцию `MPI_Cart_sub`, расщепить матрицу процессов на три одномерных столбца (при этом процессы 0, 1 и 2 будут главными процессами в полученных столбцах). Используя одну коллективную операцию пересылки данных, переслать по одному исходному числу из главного процесса каждого столбца во все процессы этого же столбца и вывести полученное число в каждом процессе (включая процессы 0, 1 и 2).

Выполнение работы.

Реализация началась с инициализации MPI в функции `main`: вызов `MPI_Init` для запуска среды, получение ранга (`MPI_Comm_rank`) и общего числа процессов (`MPI_Comm_size`).

Был определен параметр `Multiplicity = 3`, который описывает кратность задачи - количество столбцов в создаваемой декартовой топологии. Этот параметр используется во всей программе для обеспечения соблюдения условия кратности числа процессов трем.

Перед выполнением основной логики добавлена проверка корректности количества процессов с помощью функции `isValidCountOfProcesses`, которая использует параметр `Multiplicity` для гарантии, что общее число процессов кратно трем и больше трех, как требуется по условию задачи.

Основная логика программы была вынесена в функцию `processTask`. В начале функции вычисляется параметр $N = \text{world_size} / \text{Multiplicity}$, определяющий количество строк в декартовой топологии.

Далее следует ключевая операция работы - вызов `MPI_Cart_create`. Эта функция создает виртуальную декартову топологию в виде матрицы размером $N \times \text{Multiplicity}$. Параметры топологии:

- `dims = {N, Multiplicity}` - размеры сетки по измерениям (строки $\times$ столбцы), где `Multiplicity` определяет количество столбцов
- `periods = {0, 0}` - нециклическая сетка
- `reorder = 0` - сохранение исходной нумерации процессов

После создания декартовой топологии каждый процесс определяет свои координаты в сетке с помощью `MPI_Cart_coords`. Координата `my_row` соответствует номеру строки (от 0 до $N-1$), координата `my_col` - номеру столбца (от 0 до $\text{Multiplicity}-1$).

Следующий важный этап - использование функции `MPI_Cart_sub` для расщепления матрицы процессов на одномерные группы. Параметр `remain_dims = {1, 0}` используется для определения сохраняемых размерностей. В результате получаются `Multiplicity` (3) групп процессов (по количеству столбцов), каждая размером равным N (количеству строк). Каждая такая группа представляет собой вертикальный столбец исходной матрицы.

В каждом созданном подкоммуникаторе процессы определяют свой локальный ранг и размер группы. Проверка с помощью `assert` гарантирует, что размер подкоммуникатора равен N . Главными процессами в каждом столбце являются процессы с координатой `my_row = 0` (первая строка исходной матрицы), что соответствует процессам 0, 1 и 2 в глобальной нумерации, как и требуется по условию задачи.

Для подготовки данных используется функция `generateData`, которая создает массив из N целых чисел для главных процессов. Числа формируются по формуле `my_col * 100 + i`, что позволяет легко идентифицировать столбец-источник данных.

Основная коллективная операция программы - `MPI_Scatter` - выполняется внутри каждого подкоммуникатора-столбца. Главный процесс столбца (с `my_row = 0`) рассылает по одному числу из своего массива всем процессам в

столбце, включая себя. Таким образом, каждый процесс получает одно число от главного процесса своего столбца.

Завершается функция выводом полученных значений с указанием глобального ранга процесса, его координат в декартовой топологии, локального ранга в подкоммуникаторе и полученного числа. После выполнения основной логики созданные коммуникаторы освобождаются с помощью `MPI_Comm_free`.

Для анализа производительности добавлены функции замера времени выполнения, которые собирают статистику по всем процессам и выводят максимальное и среднее время выполнения на процессе с рангом 0.

Для визуального представления алгоритма нарисована сеть Петри для 6 процессов представленная на рисунке 1.

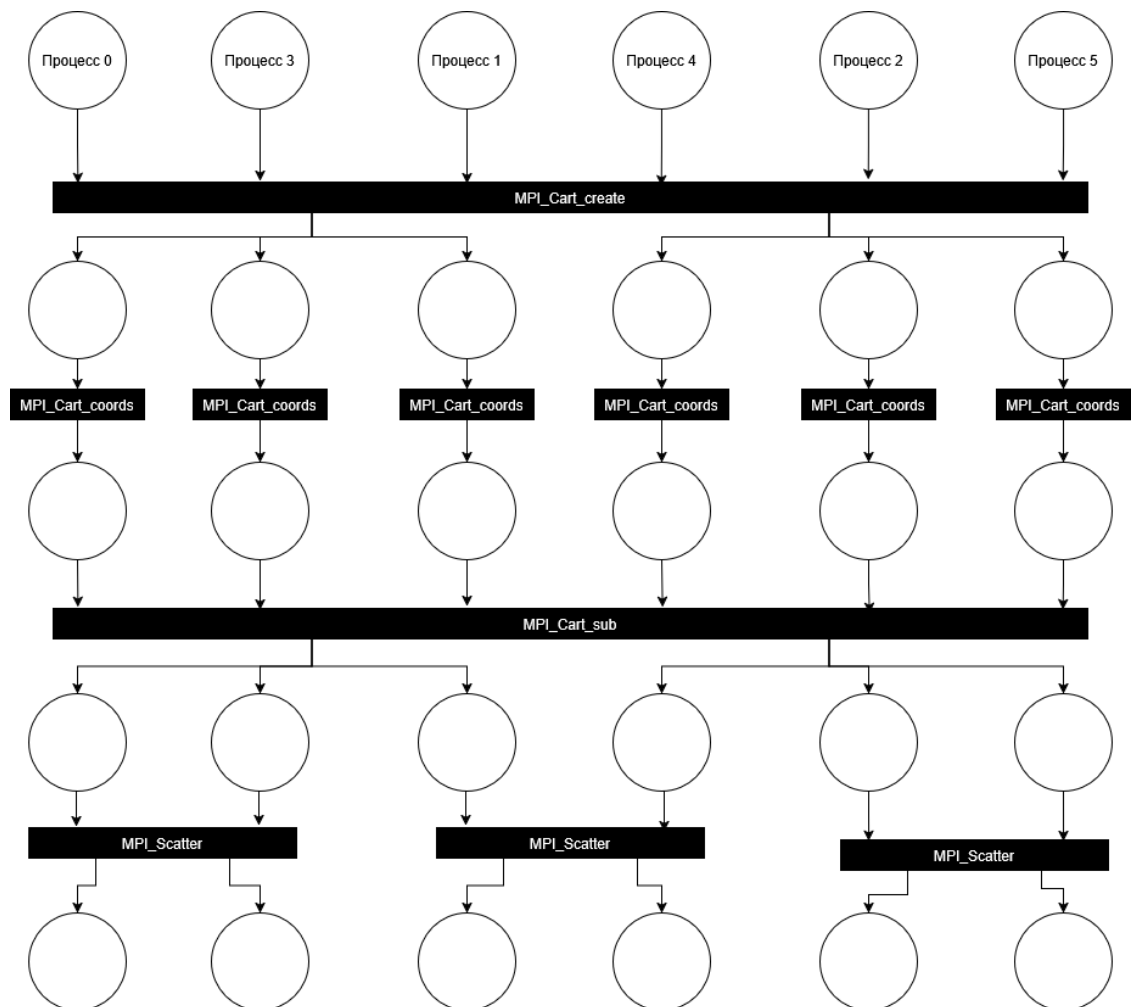


Рисунок 1 – Сеть Петри для 6 процессов.

Исследуем время выполнения программы при различном размере задачи для различного числа процессов. Результаты экспериментов представлены в

таблице 1. Для каждого числа процессов значения приведены в усреднение на 50 запусков программы (время работы программы считалось по среднему времени завершения всех процессов).

Таблица 1.

Сложность задачи	Число процессов								
	6	9	12	15	18	21	24	27	30
0	11.73	11.75	18.42	28.05	35.74	46.61	51.69	69.07	111.58

По данным из таблицы можно сделать несколько ключевых выводов о поведении программы с декартовой топологией и операцией MPI_Scatter.

Во-первых, как видно из рисунка 2, при малом количестве процессов (6-9) время остается практически неизменным, что свидетельствует об эффективном использовании вычислительных ресурсов. Однако при дальнейшем увеличении числа процессов начинается монотонный рост времени выполнения.

Этот рост объясняется увеличением накладных расходов, связанных с организацией параллельных вычислений. С увеличением количества процессов возрастают затраты времени на синхронизацию, переключение контекста и организацию коммуникаций между процессами. Особенно заметно влияние этих факторов при большом количестве процессов, где накладные расходы начинают преобладать над полезной вычислительной работой.

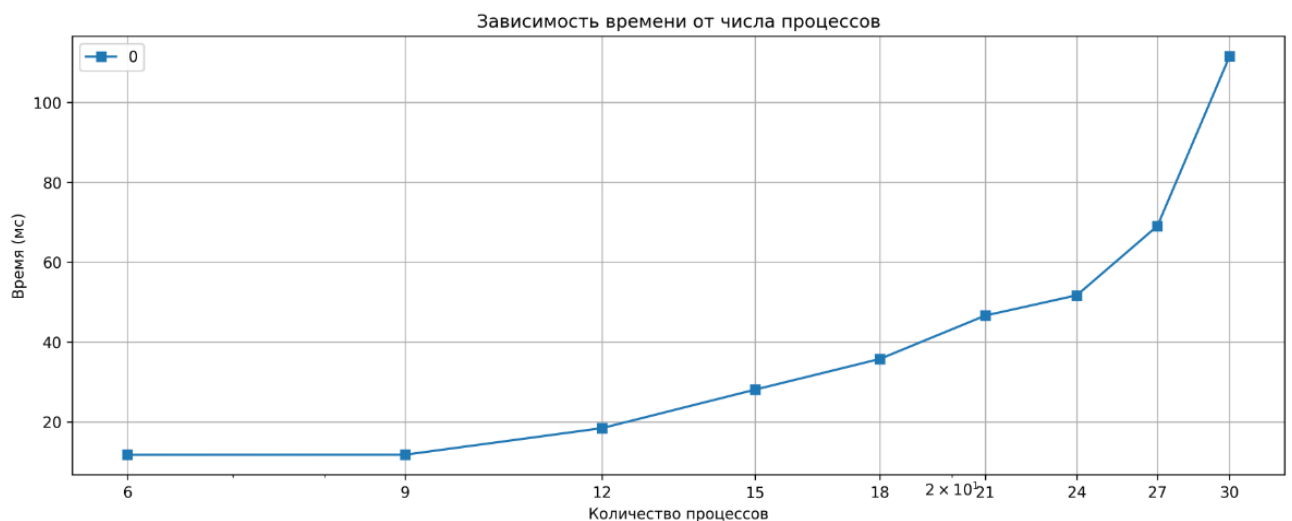


Рисунок 2. Зависимость времени от числа процессов.

Рассчитаем ускорение программы по формуле:

$$S_p(n) = \frac{T_1(n)}{T_p(n)}, \text{ где } p - \text{число процессов, а } n - \text{размеры поля}$$

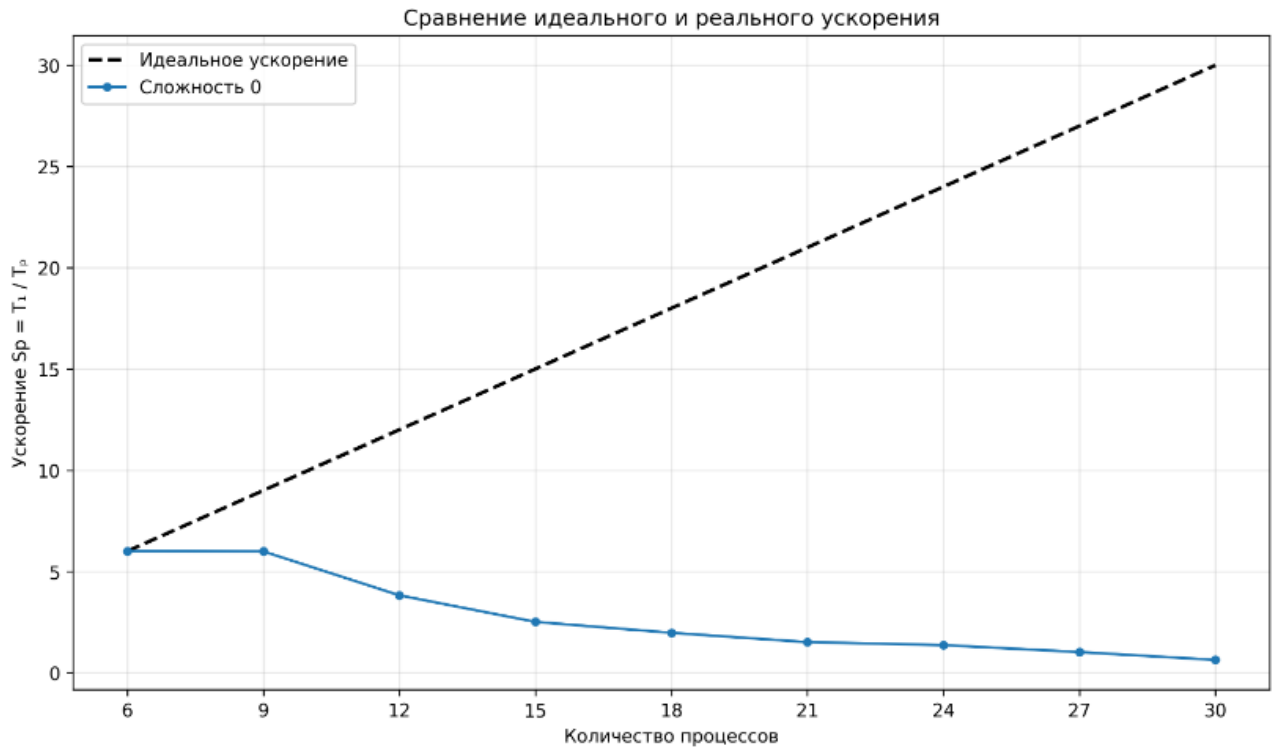


Рисунок 3. Ускорение программы для разной сложности задачи при увеличении числа процессов.

Как видно из рисунка 3, наблюдается выраженное замедление работы программы с ростом числа процессов. Время выполнения увеличивается более чем в 9 раз при переходе от 6 к 30 процессам.

Эффект ускорения отсутствует - вместо уменьшения времени выполнения с ростом числа процессов наблюдается обратная тенденция. Это свидетельствует о том, что для данной задачи параллельная реализация не обеспечивает выигрыша в производительности при использовании большого количества процессов.

По сути, в данной программе отсутствует настоящее распараллеливание вычислений - основное время работы занимают не вычисления, а накладные расходы на организацию параллельной инфраструктуры. Создание декартовой топологии, работа с коммутаторами и синхронизация процессов требуют

значительных затрат времени, которые не компенсируются вычислительной работой. В результате с ростом числа процессов увеличиваются именно эти накладные расходы, а не ускоряются вычисления, что и приводит к замедлению работы программы.

Исходный код программы находится в приложении А, результат выполнения программы для 9 процессов находится в приложении Б.

Выводы.

В ходе лабораторной работы была разработана программа на C++ с использованием MPI, демонстрирующая создание виртуальной декартовой топологии с помощью функции `MPI_Cart_create` и последующее разделение на подкоммуникаторы с использованием `MPI_Cart_sub`. Была построена матричная топология размером $N \times 3$, после чего процессы были сгруппированы в столбцы, где главными процессами стали процессы 0, 1 и 2. С помощью коллективной операции `MPI_Scatter` выполнена рассылка данных от главных процессов ко всем процессам в соответствующих столбцах. Полученные значения корректно выводятся каждым процессом с указанием его координат в декартовой сетке, что подтверждает правильность работы с виртуальной топологией.

Анализ производительности показал, что программа демонстрирует замедление работы с ростом числа процессов. При использовании 6-9 процессов время выполнения остается стабильным, однако дальнейшее увеличение количества процессов приводит к монотонному росту времени выполнения. Это объясняется преобладанием накладных расходов над полезными вычислениями - затраты на создание и управление декартовой топологией, организацию подкоммуникаторов и синхронизацию в операции `MPI_Scatter` оказываются значительными по сравнению с объемом передаваемых данных. Особенно заметен рост накладных расходов при большом количестве процессов, где коммуникационные издержки начинают доминировать.

Таким образом, работа подтвердила, что эффективное использование виртуальных топологий в MPI требует тщательного баланса между вычислительной нагрузкой и коммуникационными операциями. Для задач с малым объемом вычислений создание сложных топологий может оказаться нецелесообразным из-за преобладания накладных расходов на синхронизацию и управление коммутаторами. Оптимальная производительность достигается при использовании минимального необходимого количества процессов, соответствующего структуре решаемой задачи.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

```
main.cpp
#include <mpi.h>
#include <iostream>
#include <vector>
#include <cassert>
#include <algorithm>
#include <numeric>
#include <fstream>

#define Multiplicity 3

bool isValidCountOfProcesses(int world_size, int world_rank) {
    return world_size <= Multiplicity || world_size % Multiplicity != 0;
}

std::vector<int> generateData(int my_col, int N) {
    std::vector<int> data(N);
    for (int i = 0; i < N; ++i) {
        data[i] = my_col * 100 + i;
    }
    return data;
}

void processTask(int world_size, int world_rank) {
    int N = world_size / Multiplicity;

    int dimSize = 2;
    int dims[dimSize] = {N, Multiplicity};
    int periods[dimSize] = {0, 0};
    int reorder = 0;
    MPI_Comm cart_comm;
    MPI_Cart_create(MPI_COMM_WORLD, dimSize, dims, periods, reorder,
&cart_comm);

    int coords[dimSize];
    MPI_Cart_coords(cart_comm, world_rank, dimSize, coords);
    int my_row = coords[0];
    int my_col = coords[1];

    int remain_dims[dimSize] = {1, 0}; // keep rows, drop cols => получим
КОММ. по столбцам
    MPI_Comm col_comm;
    MPI_Cart_sub(cart_comm, remain_dims, &col_comm);

    int sub_rank, sub_size;
    MPI_Comm_rank(col_comm, &sub_rank);
    MPI_Comm_size(col_comm, &sub_size);
    assert(sub_size == N);

    int root_in_subcomm = 0;
    std::vector<int> sendbuf;
    int recv_value = 0;
```

```

bool is_global_main = (my_row == 0);
if (is_global_main) {
    sendbuf = generateData(my_col, N);
}

MPI_Scatter(
    is_global_main ? sendbuf.data() : nullptr,
    1,
    MPI_INT,
    &recv_value,
    1,
    MPI_INT,
    root_in_subcomm,
    col_comm
);

std::cout << "GlobalRank=" << world_rank
    << " (row=" << my_row << ", col=" << my_col << ") "
    << "sub_rank=" << sub_rank
    << " received=" << recv_value << std::endl;

MPI_Comm_free(&col_comm);
MPI_Comm_free(&cart_comm);
}

void getTimeStatistic(int rank, int size, double processTime) {
    std::vector<double> allTimes;
    if (rank == 0) {
        allTimes.resize(size);
    }
    MPI_Gather(&processTime, 1, MPI_DOUBLE,
        allTimes.data(), 1, MPI_DOUBLE,
        0, MPI_COMM_WORLD);

    if (rank == 0) {
        double maxTime = *std::max_element(allTimes.begin(),
allTimes.end());
        double avgTime = std::accumulate(allTimes.begin(),
allTimes.end(), 0.0) / size;

        std::cout << "\n=== РЕЗУЛЬТАТЫ ДЛЯ " << size << " ПРОЦЕССОВ ==="
<< std::endl;
        std::cout << "Максимальное время: " << maxTime << " мс" <<
std::endl;
        std::cout << "Среднее время: " << avgTime << " мс" << std::endl;

        std::ofstream file("results.txt", std::ios::app);
        if (file.is_open()) {
            file << size << " " << maxTime << " " << avgTime <<
std::endl;
            file.close();
            std::cout << "Результаты записаны в results.txt" <<
std::endl;
        } else {
            std::cout << "Ошибка открытия файла!" << std::endl;
        }
    }
}

```

```

    }
}

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);

    int world_size, world_rank;
    MPI_Comm_size(MPI_COMM_WORLD, &world_size);
    MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);

    if (isValidCountOfProcesses(world_size, world_rank)) {
        if (world_rank == 0) {
            std::cerr << "Error: the number of processes must be a
multiple of " << Multiplicity << " and > " << Multiplicity << ". Current
number of processes = "
                        << world_size << std::endl;
        }
        MPI_Finalize();
        return 0;
    }

    double startTime = MPI_Wtime();
    processTask(world_size, world_rank);
    double endTime = MPI_Wtime();
    double processTime = (endTime - startTime) * 1000;
    MPI_Barrier(MPI_COMM_WORLD);

    getTimeStatistic(world_rank, world_size, processTime);

    MPI_Finalize();
    return 0;
}

```

ПРИЛОЖЕНИЕ Б

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ ПРОГРАММЫ

```
GlobalRank=4 (row=1, col=1) sub_rank=1 received=101
GlobalRank=5 (row=1, col=2) sub_rank=1 received=201
GlobalRank=6 (row=2, col=0) sub_rank=2 received=2
GlobalRank=7 (row=2, col=1) sub_rank=2 received=102
GlobalRank=8 (row=2, col=2) sub_rank=2 received=202
GlobalRank=0 (row=0, col=0) sub_rank=0 received=0
GlobalRank=1 (row=0, col=1) sub_rank=0 received=100
GlobalRank=2 (row=0, col=2) sub_rank=0 received=200
GlobalRank=3 (row=1, col=0) sub_rank=1 received=1
```